Úa: Using the compiled uvh assembly

Hilmar Gudmundsson

26 October 2026

Contents

1 Overview 1

2 Quick start on Windows 2

3 Recommended settings 2

4 What is supported, and the automatic fallback 3

5 Checking that it is used 4

6 Building the MEX files yourself on Windows 5

7 When to rebuild 6

8 Threads, parallel pools and shared machines 7

9 Linux 7

10 Troubleshooting 8

11 Regression test 10

12 Performance 10

1 Overview

Setting `CtrlVar.uvhMatrixAssembly.CompiledKernel=true` makes Úa assemble the uvh system (the residual and the Jacobian of the implicit uvh time step) with compiled code instead of vectorised MATLAB code. The results are the same to rounding errors, and the solver takes the same time steps and Newton iterations.

The compiled code is generated from Úa's own MATLAB functions with MATLAB Coder, and runs in parallel over the elements on all processor cores (OpenMP). It is a drop-in replacement: no other option, input file or physics changes.

- **Default:** off (`CompiledKernel=false`), so nothing changes unless you switch it on.
- **Scope:** only the uvh assembly. The uv assembly (diagnostic step), the level-set equation and the rest of Úa are unchanged.

- **Fallback:** if an option or element type is not supported, or the MEX files are missing, outdated or fail when called, Úa uses the vectorised assembly instead, and says why once per session.
- **Tested:** a regression test (`runtests("TestUaCompiled")`, 40 tests, about 25 s) compares the compiled with the vectorised assembly for all options (Section 11).
- **When it pays off:** for any transient run, and most for medium and large meshes. The uvh assembly becomes 6 to 16 times faster, and whole runs 1.4 to 4 times faster (see Section 12).

The compiled code lives in `UaSource/UaCompiled`. Ready-built MEX files for Windows (64-bit) are included; on Linux you build them once yourself (Section 9).

How the compiled assembly works, and what to observe when changing the code it is built from, is described in the separate document *Úa: Using the compiled uvh assembly – Notes for Developers*.

2 Quick start on Windows

On 64-bit Windows with MATLAB R2026b, the included MEX files work as they are: you need neither MATLAB Coder nor a C compiler. Two lines in `DefineInitialInputs.m` switch the compiled assembly on.

1. Put Úa on the path as usual. `UaSource/UaCompiled` is included by `genpath`:

```
addpath(genpath('C:\...\UaSource'))
```

2. In `DefineInitialInputs.m`, add:

```
CtrlVar.uvhMatrixAssembly.CompiledKernel=true;
CtrlVar.Parallel.uvhAssembly.spmd.isOn=false;   % the compiled assembly is
    parallel already
```

3. Run Úa as usual. Nothing else changes, and no message appears while the compiled assembly is used.

You can also switch it on from a run script, without editing `DefineInitialInputs.m`:

```
CtrlVarOnInput.uvhMatrixAssembly.CompiledKernel=true;
CtrlVarOnInput.Parallel.uvhAssembly.spmd.isOn=false;
Ua(UserVar,CtrlVarOnInput)
```

The included MEX files were built with MATLAB R2026b. MEX files generated by MATLAB Coder generally only run in the release they were built with, so with another release, build them yourself (Section 6).

3 Recommended settings

Use the compiled uvh assembly without spmd for the uvh assembly; for large meshes, keep spmd for the uv assembly; and use PARDISO as the linear solver if you can.

Setting	Recommended	Why
`CtrlVar.uvhMatrixAssembly.CompiledKernel`	`true`	Switches the compiled uvh assembly on (default `false`).
`CtrlVar.Parallel.uvhAssembly.spmd.isOn`	`false`	With spmd on, the uvh assembly runs on the spmd workers, and the compiled kernel is not used there at all. The compiled assembly is parallel already, and faster than spmd.
`CtrlVar.Parallel.uvAssembly.spmd.isOn`	`true` for large meshes (from about 50,000 elements), with a parallel pool; otherwise `false`	The uv assembly is not compiled. With spmd it is about 2 times faster on large meshes; on small meshes spmd costs more than it saves.
`CtrlVar.LinearSolver.Type`	`"PARDISO"`, if the Intel MKL runtime is available	Not part of the compiled assembly, but once the assembly is fast, the linear solves take the largest share of the time. See `Ua2D_DefaultParameters.m` for the setup.

For a large run (Greenland, 110,000 elements), this combination took 189 s against 779 s for the vectorised assembly with spmd and MATLAB's linear solver.

With spmd for the uv assembly, create the pool before the run, for example in `DefineInitialInputs.m`:

```
if isempty(gcp("nocreate")) ; parpool("Threads",4) ; end
```

4 What is supported, and the automatic fallback

The compiled assembly supports all options of the forward uvh assembly; where it cannot be used, Úa falls back to the vectorised assembly and prints the reason once per MATLAB session.

Supported, all chosen at run time (no rebuild needed when they change):

- all numeric parameters in `CtrlVar`, including the time step;
- all sliding laws, with all their names and aliases (see `SlidingLawCode.m`);
- all SUPG tau options: `"tau1"`, `"tau2"`, `"taus"`, `"taut"`, `"tau0"`;
- the thickness penalty (softplus, polynomial, exponential), the thickness barrier, `ResetThicknessToMinThickness`, `Enforce_bAboveB`;
- level-set mass-balance feedback, and the mélange terms;
- `OnlyCalcBasalDragAndEffectiveViscosity` (used by `CalcBasalTraction.m`);
- linear, quadratic and cubic elements, with quadrature rules of up to 37 points.

The diagnostic plots and output inside the assembly (`doplots` with `PlotSUPGparameter`, and `InfoLevelThickMin>=10`) are not produced by the compiled code. To see them, switch the

compiled assembly off for that run.

Situation	What happens	What to do
`CtrlVar.Parallel.uvhAssembly.spmd.isOn=true`	The vectorised assembly runs on the spmd workers. No message.	Set it to `false`.
`UaSource/UaCompiled` not on the path	Vectorised assembly; message: *. . . UaSource/UaCompiled is not on the path. . .*	Add it, eg with `addpath(genpath(UaSource))`.
MEX files not found (another operating system, not built yet)	Vectorised assembly; the message names the missing MEX files.	Build them (Sections 6 and 9).
Elements with more than 10 nodes, or quadrature rules with more than 37 points	Vectorised assembly; the message names the element type.	None needed; rare.
Mesh without the cached assembly pattern (`CtrlVar.MUA.AssemblyPattern.uvh=false`)	Vectorised assembly; the message names the pattern.	Keep the default (`true`).
Unknown sliding law or tau option, or a `CtrlVar` field missing (eg an old restart file)	Vectorised assembly; the message names the options.	Correct the option, or check `CtrlVar` against `Ua2D_DefaultParameters.m`.
MEX files outdated: a source file changed since they were built, or built with another MATLAB release (Section 7)	Vectorised assembly; the message names the changed files or the release.	Rebuild them (Section 6).
A MEX file fails when called (eg *Invalid MEX-file*, or inputs that no longer match)	Vectorised assembly for the rest of the MATLAB session; the message, printed once, gives the error.	Rebuild them; then `clear uvhCompiledKernelCheck`, or restart MATLAB.

5 Checking that it is used

With the option on and spmd off for the uvh assembly, no message means the compiled assembly is used. To confirm it explicitly:

- **After (part of) a run,** list the loaded kernel MEX files. For linear elements with 6 integration points you should see `uvhKernelFull_n3i6_mex` and `uvhKernelRonly_n3i6_mex`, otherwise `uvhKernelFull_mex` and `uvhKernelRonly_mex`:

```
[~,m]=inmem; m(contains(m,"uvhKernel"))
```

- **With the profiler,** `uvhMatrixAssemblyCompiled` and the kernel MEX files appear in the profile, and `uvhMatrixAssemblySSTREAM_SPMD` or the loop over integration points does not.

- **Before a long run,** for a given `CtrlVar` and mesh:

```
[ok,~,KernelFull]=uvhCompiledKernelCheck(CtrlVar,MUA)
```

`ok=true` means the compiled assembly will be used, with the MEX file named in `KernelFull`; otherwise the reason is printed.

If you want to see for yourself that the results are the same, run a few time steps with `CompiledKernel=false` and `true` and compare: the solutions agree to rounding errors (in our tests, ice thickness to about 10^{-11} m), with the same numbers of time steps and Newton iterations.

6 Building the MEX files yourself on Windows

You only need to build the MEX files if you use a MATLAB release other than R2026b, or after changes to the code they are built from (Section 7). Building takes about 4 minutes and needs MATLAB Coder and a C compiler with OpenMP.

Requirements

- MATLAB Coder (a separate licence).
- A C compiler that MATLAB supports, with OpenMP. Tested: MinGW-w64 14.2, installed through the Add-On Explorer as *MATLAB Support for MinGW-w64 C/C++/Fortran Compiler*. Microsoft Visual C++ should also work (the build script switches on OpenMP for it), but has not been tested.

Steps

1. Select the C compiler once:

   ```
   mex -setup C
   ```

2. Build in a **fresh** MATLAB session. A MEX file that has been called in a session stays loaded and cannot be replaced, so start the build before running Úa, or from the command line:

   ```
   matlab -batch "addpath(genpath('C:\...\UaSource')); BuildUaCompiled"
   ```

3. `BuildUaCompiled` writes five MEX files to `UaSource/UaCompiled`, and the build information `UaCompiledBuildInfo_mexw64.mat` (the MATLAB release and checksums of the source files of each MEX file, see Section 7); the intermediate files go to the temporary folder.

4. Check the build with the regression test (Section 11):

   ```
   results=runtests("TestUaCompiled"); table(results)
   ```

MEX file	Used for
`uvhKernelFull_n3i6_mex`, `uvhKernelRonly_n3i6_mex`	Linear elements with 6 integration points (fixed sizes, the fastest)
`uvhKernelFull_mex`, `uvhKernelRonly_mex`	All other element types (up to 10 nodes and 37 integration points)
`SparseFromCSC`	Creating the sparse Jacobian directly (hand-written C)

`Full` computes the residual and the Jacobian, `Ronly` the residual only (used in the line searches). To build only some of them, list them, for example for linear elements only:

```
BuildUaCompiled(["Full_n3i6","Ronly_n3i6","Sparse"])
```

The resulting MEX files depend only on MATLAB's own libraries and the Windows runtime, not on the compiler, so they can be copied to other Windows computers with the same MATLAB release. Copy `UaCompiledBuildInfo_mexw64.mat` with them: without it, Úa cannot check whether they are up to date, and says so.

7 When to rebuild

The MEX files contain a compiled copy of the MATLAB functions listed below. Rebuild them whenever one of these changes, or when you change MATLAB release; options and parameters never need a rebuild.

Rebuild after:

- **a change of MATLAB release** (MEX files generated by MATLAB Coder generally only run in the release they were built with);
- **an update of Úa** (eg `git pull`) that changes any of these files:
  - in `UaCompiled`: `uvhElementKernel.m`, `uvhKernelFull.m`, `uvhKernelRonly.m`, `SparseFromCSC.c`, and the list of fields in `MakeKernelParamsFlex.m`;
  - in `UaMain`: `BasalDrag.m`, `SlidingLawBeta.m`, `SlidingLawCode.m`, `EffectiveViscositySSTREAM.m`, `SUPGtau.m`, `SUPGtauCode.m`, `ThicknessPenaltyMassBalanceFeedback.m`, `LevelSetMethodMassBalanceFeedback.m`, `HeavisideApprox.m`, `DiracDelta.m`, `SmoothFloor.m`, `SoftPlus.m`, `shape_fun.m`.

To see whether an update touched them, for example:

```
git diff --stat <old commit> HEAD -- UaCompiled UaMain/BasalDrag.m UaMain/
    SlidingLawBeta.m UaMain/SUPGtau.m ...
```

No rebuild is needed for changes to options or parameters (sliding law, tau option, penalty function, time step and so on), or to the other MATLAB files in `UaCompiled` (`uvhMatrixAssemblyCompiled.m`, `uvhCompiledKernelCheck.m`, `uvhCSCPattern.m`).

Úa detects outdated MEX files itself. `BuildUaCompiled` records, for each MEX file, the MATLAB release and a checksum of each of its source files in `UaCompiledBuildInfo_<mexext>.mat`; once per MATLAB session, before the first compiled assembly, `uvhCompiledKernelCheck` compares them (about 0.1 s). If a source file has changed since the build, or another MATLAB release is running, the vectorised assembly is used, with a message such as

```
uvhMatrixAssembly: compiled kernel not used, as the MEX files are outdated: UaMain/
    SoftPlus.m changed since they
were built (rebuild them with BuildUaCompiled.m). The vectorised assembly is used
    instead.
```

Changed line endings alone (eg from a git checkout) do not count as a change. Without build information (eg MEX files built before 26 October 2026), a note says that this could not be

checked, and the MEX files are used.

8 Threads, parallel pools and shared machines

By default the compiled assembly uses one thread per logical processor (32 threads on a 16-core processor with hyperthreading); limit this with the environment variable `OMP_NUM_THREADS`, set before MATLAB starts.

Threads	Kernel time per full assembly (Greenland, 197,000 elements)
1	1,992 ms
8	394 ms
16	295 ms
32 (default)	269 ms

- **Setting the number of threads.** The OpenMP runtime reads `OMP_NUM_THREADS` once, when the compiled code is first called, so set it before starting MATLAB, for example in a Windows command window:

  ```
  set OMP_NUM_THREADS=8
  matlab
  ```

  `maxNumCompThreads` in MATLAB does not affect the compiled assembly.

- **Shared computers.** Set `OMP_NUM_THREADS` to the number of cores you are allowed to use.

- **Several Úa runs at the same time** (separate MATLAB processes, or `batch` jobs): each run would start one thread per logical processor, and they would compete. Give each run its share, for example 4 runs on 16 cores: `OMP_NUM_THREADS=4` in each.

- **Parallel pools.** A pool for the uv assembly (`CtrlVar.Parallel.uvAssembly.spmd.isOn=true`) is no problem: its workers are idle while the uvh assembly runs. The compiled assembly is never used inside spmd workers; with `CtrlVar.Parallel.uvhAssembly.spmd.isOn=true` the vectorised assembly is used instead (Section 3).

- **Running Úa inside `parfor` or on the workers of a pool** (eg ensembles) has not been tested with the compiled assembly. If you do, limit the threads per worker with `OMP_NUM_THREADS`.

9 Linux

On Linux you build the MEX files once yourself; after that, everything works as on Windows. The included MEX files are for Windows only (`.mexw64`); on Linux MATLAB looks for `.mexa64` files, so until they are built, Úa uses the vectorised assembly and says that the MEX files were not found.

The compiled assembly has so far only been tested on Windows. The steps below are those expected for Linux; check the result as described at the end of this section.

Requirements

- MATLAB Coder.

- A GCC version supported by your MATLAB release (see MathWorks' list of supported compilers). GCC includes OpenMP.

Steps

1. Check that the C compiler is selected (usually `gcc` already is):

   ```
   mex -setup C
   ```

2. Build in a fresh session:

   ```
   matlab -batch "addpath(genpath('/path/to/UaSource')); BuildUaCompiled"
   ```

   This writes `uvhKernelFull_mex.mexa64`, `uvhKernelRonly_mex.mexa64`, the two `_n3i6_` variants, `SparseFromCSC.mexa64` and the build information `UaCompiledBuildInfo_mexa64.mat` to `UaSource/UaCompiled`. For compilers it does not recognise, `BuildUaCompiled` builds `SparseFromCSC` without OpenMP and says so; it then still works, only copies serially.

3. Switch the compiled assembly on as on Windows (Section 2, step 2).

Checking the build. Run the regression test (Section 11); all 40 tests should pass:

```
results=runtests("TestUaCompiled"); table(results)
```

For your own setup, run `uvhCompiledKernelCheck(CtrlVar,MUA)` (Section 5), and a few time steps with `CompiledKernel=false` and `true`: the solutions should agree to rounding errors, with the same numbers of Newton iterations.

Clusters

- Build once per MATLAB release, on a node with the same operating system and processor type as the compute nodes; the `.mexa64` files can then be used on all of them through a shared `UaSource`.
- Do not rebuild while runs are using the MEX files.
- Give each job its cores through `OMP_NUM_THREADS`, for example with SLURM:

  ```
  #SBATCH --cpus-per-task=16
  export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK
  matlab -batch "run('MyUaRun.m')"
  ```

- The `.mexa64` files and `UaCompiledBuildInfo_mexa64.mat` are not in the git repository. Keep them out of commits from Linux machines (or add `*.mexa64` and `UaCompiled/UaCompiledBuildInfo_mexa64.mat` to `.gitignore`).

macOS is not covered: Apple's compiler has no OpenMP, and the build has not been tried there.

10 Troubleshooting

Most problems come from MEX files that are missing, outdated or built for another MATLAB release. Úa detects these and uses the vectorised assembly, with a message saying why;

rebuilding the MEX files removes the cause. Switching the option off (`CompiledKernel=false`) always gets a run going again.

Symptom	**Cause**	**Remedy**
Message *uvhMatrixAssembly: compiled kernel not used, as ...*	An option, element type or file that the compiled assembly cannot handle; the message says which.	See Section 4. The run continues with the vectorised assembly.
No speed-up, and no message	`CtrlVar.Parallel.uvhAssembly.spmd.isOn=true`: the vectorised assembly runs on the spmd workers.	Set it to `false`.
Message *compiled kernel not used, as the MEX files are outdated: ...* or *... built with another MATLAB release ...*	A source file of the MEX files changed since they were built (eg after `git pull`), or another MATLAB release.	Rebuild them (Section 6). Until then, the vectorised assembly is used.
Message *the compiled kernel failed (... Invalid MEX-file ...)*, or an error about the class, size or fields of an input	MEX files built for another operating system, damaged, or older than the current `uvhElementKernel.m` or `MakeKernelParamsFlex.m`.	Rebuild them. The vectorised assembly is used for the rest of the session.
Built the MEX files, but Úa still says they were not found, outdated or failed	Úa checks once per session whether the MEX files exist and are up to date, and remembers the answer (and a failure).	`clear uvhCompiledKernelCheck`, or restart MATLAB.
`BuildUaCompiled` fails to write a MEX file	The MEX file was called in this session and is still loaded (MEX files with OpenMP are not released by `clear mex`).	Restart MATLAB and build first, or use `matlab -batch` (Section 6).
`BuildUaCompiled`: *No C compiler selected*	No compiler, or none selected.	Install one (Windows: MinGW-w64 add-on), then `mex -setup C`.
`BuildUaCompiled` reports an error in a UaMain function (eg after editing `BasalDrag.m`)	The change uses something MATLAB Coder does not support.	See the rules in *Úa: Using the compiled uvh assembly – Notes for Developers*; the error report names the line.
The computer is slow for other work during a run, or parallel runs are slower than expected	The compiled assembly uses all logical processors.	Limit it with `OMP_NUM_THREADS` (Section 8).
Results differ by more than rounding errors from `CompiledKernel=false`	Not expected.	Run the regression test (Section 11); if it passes but your results differ, report it with the `CtrlVar` settings.

11 Regression test

A regression test checks that the compiled assembly gives the same results as the vectorised one, for all supported options. Run it after building the MEX files, after updating Úa, or if in doubt:

```
results=runtests("TestUaCompiled");    % 40 tests, about 25 s
table(results)
```

All tests should pass. The test (`UaSource/UaCompiled/Tests/TestUaCompiled.m`) needs no input files: it builds a small synthetic ice sheet itself (grounded and floating ice, a calving front, thin ice near `ThickMin`), and compares the residual and the Jacobian of both assemblies for all sliding laws, SUPG tau options, thickness penalty functions, element types and quadrature rules, and other options; it also solves one implicit time step with both. For each case it first checks that the compiled path is actually used. A failing test names the case and the quantity, eg *sliding law Weertman, ...: R, thickness rows*, with the difference found.

If tests fail because the MEX files are missing or outdated, build them first (Section 6). If they fail with up-to-date MEX files, please report it.

12 Performance

The compiled uvh assembly is 6 to 16 times faster than the vectorised assembly, depending on the mesh and on whether the vectorised assembly uses spmd; whole runs became 1.4 to 4 times faster. Measured on a 16-core Windows desktop, MATLAB R2026b; your figures will differ with the machine.

Case	Vectorised	Compiled	Speed-up
One full uvh assembly, Greenland, 197,000 elements	3,903 ms (serial), 1,681 ms (spmd, 4 threads)	244 ms	16× (serial), 6.9× (spmd)
One residual-only assembly, same mesh	1,433 ms (serial), 477 ms (spmd)	93 ms	15×, 5.1×
One full uvh assembly, Thule, 12,000 elements	175 ms (serial)	23 ms	7.6×
Thule run, 0.1 yr, mesh adaptation and level set: uvh assembly / whole run	91 s / 209 s (serial)	17 s / 146 s	5.4× / 1.4×
Greenland run, 197,000 elements, 0.1 yr	189 s (spmd for uvh and uv)	133 s (spmd for uv only)	1.4×
Greenland run, 110,000 elements, 1 yr: uvh assembly / whole run	245 s / 779 s (spmd, MATLAB solver)	38 s / 189 s (PARDISO)	6.5× / 4.1×

The gain grows with the mesh size and with the number of cores. In the last case, about two thirds of the saving came from PARDISO and one third from the compiled assembly: once the assembly is fast, the linear solves and the uv assembly take most of the remaining time.